ROBOTIC VEHICLE FOR SURVEILLANCE AND HAZARDOUS ZONE OPERATIONS

ABSTRACT

Border surveillance and hazardous zone monitoring are critical operations that expose security personnel to significant life-threatening risks. To minimize human involvement in such dangerous environments, this project proposes an Embedded Robotic Vehicle for Border Surveillance and Hazardous Zone Operations integrated with IoT technology and mobile-based visual monitoring. The system enables real-time remote surveillance, data transmission and alert generation to ensure continuous situational awareness. The robotic vehicle is driven by DC gear motors controlled through an L293D motor driver, ensuring smooth and controlled movement in all directions. An IoT module interfaced with the embedded controller facilitates remote access through a cloud-based application, allowing authorized users to control navigation and monitor sensor outputs from distant locations. A fire sensor is deployed to detect abnormal heat levels and fire hazards within restricted zones, triggering instant notifications for preventive action. In addition, a human detection mechanism identifies unauthorized presence near border areas to strengthen security monitoring. A mobile camera provides live visual surveillance by capturing real-time video streams for evidence-based decision-making. The proposed system delivers a cost-effective, scalable and reliable solution that enhances operational efficiency and significantly reduces direct human exposure to hazardous environments.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**VII**
	TABLE OF CONTENTS	**IX**
	LIST OF FIGURES	**XI**
	LIST OF ABBREVIATIONS	**XII**
1.	**INTRODUCTION**	**1**
	1.1 GENERAL	1
	1.2 OBJECTIVES	2
	1.3 IMPLEMENTATION	2
2.	**LITRATURE SURVEY**	**4**
3.	**SYSTEM DESIGN**	**9**
	3.1 EXISTING SYSTEM	9
	3.2 PROPOSED SYSTEM	10
	3.2.1 Process	11
	3.3 BLOCK DIAGRAM	12
	3.4 EXPLAINATION	13
4.	**HARDWARE DESCRIPTION**	**14**
	4.1 HARDWARE REQUIREMENTS	14
	4.1.1 ESP32 Microcontroller	14
	4.1.1.1 ESP32 Pin Functions	15
	4.1.1.2 ESP32 Specifications	17

4.1.2 L293d Motor Driver 18
4.1.2.1 Functionality 18
4.1.2.2 Features 19
4.1.2.3 Pin Configuration 19
4.1.2.4 Operations 20
4.1.3 Battery 20
4.1.3.1 Lithium-Ion Batteries 20
4.1.3.2 Working Principle 21
4.1.4 Power Supply Unit 22
4.1.4.1 7812 Voltage Regulator 22
4.1.4.2 7805 Voltage Regulator 23
4.1.4.3 Circuit Considerations 24
4.1.5 Ultrasonic Sensor 24
4.1.5.1 Working Principle of Ultrasonic Sensor 25
4.1.5.2 Pin Configuration 26
4.1.6 Fire Sensor 26
4.1.6.1 Key Features 27
4.1.6.2 Pin Configuration 27
4.1.7 LCD Display 28
4.1.7.1 16X2 LCD Specifications 29
4.1.8 Dc Motor 30
4.1.8.1 Working of Geared Dc Motor 31
5. SOFTWARE DESCRIPTION 33
5.1 SOFTWARE REQUIREMENTS 33

5.1.1 Embedded C 33
5.1.2 Arduino IDE 34
5.1.3 IOT Module 36
5.2 LIBRARIES 37
5.3 CONNECTING THE ARDUINO 37
5.4 PREPARING THE BOARD 38
5.5 LOADING CODE 39
6. IMPLEMENTATION & ADVANTAGES 40
6.1 IMPLENTATION 40
6.2 ADVANTAGES 41
7. SOFTWARE OUTPUT 42
7.1 ARDUINO IOT CLOUD 42
8. RESULT & DISCUSSION 47
8.1 RESULT 47
8.2 DISCUSSION 49
9. CONCLUSION 51
APPENDIX 53
REFERENCE 59

LIST OF ABBREVIATIONS

1. **IoT** – Internet of Things
2. **ESP32** – Espressif 32-bit Microcontroller

LIST OF FIGURES

FIGURE NO.	DESCRIPTION	PAGE NO.
3.1	Existing System	9
3.2	Block Diagram	12
4.1	ESP32 DevKit Full Pinout	15
4.2	L293D Motor Driver Module	18
4.3	Arduino Uno Interfacing with L298N Motor Driver Module for Dual DC Motor Control	19
4.4	18650 lithium-ion battery	21
4.5	MB102 Breadboard Power Supply Module (Power Board v2)	23
4.6	HC-SR04 Ultrasonic Sensor (Front and Back View)	25
4.7	Infrared (IR) Flame Sensor Module	27
4.8	IR Flame Sensor module with an Arduino Uno	28
4.9	LCD display unit	28
4.10	Dc Motor	31
4.11	DC Geared Motor	32
5.1	Embedded C and IoT Module	35
7.1	Arduino Iot Cloud	42
7.2	Login Device Page	42
7.3	Project User Id	43
7.4	Dashboard	44
7.5	Control and Detection	45
8.1	Output	49

3. **LCD** – Liquid Crystal Display

4. **DC** – Direct Current

5. **ADC** – Analog to Digital Converter
6. **DAC** – Digital to Analog Converter
7. **GPIO** – General Purpose Input Output
8. **UART** – Universal Asynchronous Receiver Transmitter
9. **I2C** – Inter-Integrated Circuit
10. **SPI** – Serial Peripheral Interface
11. **PWM** – Pulse Width Modulation
12. **Wi-Fi** – Wireless Fidelity
13. **BLE** – Bluetooth Low Energy
14. **IR** – Infrared
15. **IDE** – Integrated Development Environment

CHAPTER 1

INTRODUCTION

1.1 GENERAL

Effective border surveillance and hazardous zone monitoring have become essential in modern security systems. Border areas face threats such as illegal crossings, smuggling, and infiltration, while hazardous environments like industrial sites and disaster zones present dangers including fire outbreaks and structural instability. Traditional monitoring methods rely heavily on human patrols and fixed surveillance systems, which are risky, limited in coverage, and often inefficient in remote or high-risk locations.

The advancements in embedded systems, robotics, and the Internet of Things (IoT), intelligent unmanned platforms can now perform surveillance tasks remotely. Robotic vehicles equipped with sensors and wireless communication modules can navigate difficult terrains while transmitting real-time data and video to remote control centers. These technologies reduce human exposure to danger and improve response time through instant alerts and live monitoring.

This project focuses on developing an IoT-enabled embedded robotic vehicle designed for border surveillance and hazardous zone monitoring. The system integrates mobility, sensing, cloud connectivity, and live video streaming into a unified platform. By combining fire detection, human presence sensing, obstacle detection, and remote operation, the system enhances situational awareness and supports faster, safer decision-making. The design emphasizes reliability, scalability, and cost-effectiveness, making it suitable for real-world practical implementation.

1.2 OBJECTIVES

The primary goal of this project is to design and develop an intelligent, IoT-based robotic vehicle capable of performing remote surveillance in border regions and hazardous environments. The system aims to minimize direct human involvement in high-risk areas while ensuring continuous monitoring and rapid response to potential threats. By integrating sensing mechanisms, wireless communication, and controlled mobility into a single platform, the project seeks to create a reliable and efficient surveillance solution.

To achieve stable and precise movement control using DC gear motors operated through an L293D motor driver. The robotic vehicle must be capable of smooth forward, backward, and multidirectional movement across uneven terrain. The project also aims to implement internet-based connectivity for continuous monitoring and remote control through a cloud platform. Sensor integration, including fire detection and human presence identification, is intended to enhance safety and improve situational awareness. Additionally, live video streaming through a mounted camera provides visual confirmation of detected events. The overall objective is to develop a cost-effective, scalable, and practical surveillance system that demonstrates the effective application of embedded systems and IoT technologies in real-world security operations.

1.3 IMPLEMENTATION

The implementation of the project involves integrating mechanical, electronic, and communication components into a compact robotic platform. The robotic vehicle is constructed with a durable chassis and powered by DC gear motors to enable stable movement across uneven or rough surfaces. The L293D motor driver regulates the direction and operation of the motors based on signals received from the IoT-enabled microcontroller. The microcontroller acts as the

central control unit, managing sensor data acquisition, motor control, and wireless communication.

The system incorporates multiple sensors to enhance monitoring capabilities. An ultrasonic sensor is used for obstacle detection and distance measurement to prevent collisions during navigation. A fire detection sensor identifies abnormal temperature variations or flame presence, while a human presence sensor detects movement in restricted or hazardous areas. A mobile camera module is mounted on the robotic vehicle to provide real-time video streaming for remote observation. The microcontroller establishes Wi-Fi connectivity and transmits sensor data and video feed to a cloud-based IoT platform. Remote users can access the system through a mobile or web interface to monitor conditions, receive alerts, and control the robot's movement in real time. The entire system operates in a continuous monitoring loop, ensuring uninterrupted surveillance, rapid alert generation, and efficient remote operation.

CHAPTER 2

LITERATURE SURVEY

[1] Hasib A., Hossain Apu S., Rahman A., Rana M., & Hossen R. (2025). "An IoT-Integrated Autonomous Fire Extinguishing and Surveillance Robot Development Using Smart Camera Module," International Journal of Intelligent Systems and Applications in Engineering, Vol. 13, No. 2, pp. 88–96.

The system incorporates flame and temperature sensors, a smart camera module for real-time visual monitoring and a fire suppression mechanism to control small-scale fires. IoT connectivity enables continuous monitoring and alert transmission to remote control units, improving emergency response time and operational safety, although limitations exist in handling large-scale fires and complex environmental conditions.

[2] Jebastine J., & Nanammal V. (2024). "Enhancing Wireless Surveillance Robot with Real-Time Feedback with AI & IoT Integration," Journal of Artificial Intelligence and Embedded Systems, Vol. 9, No. 3, pp. 67–75.

Jebastine and Nanammal proposed an AI and IoT integrated surveillance robot that enhances intelligent decision-making through image processing and real-time communication. The system performs motion detection, object recognition, and intrusion analysis while transmitting continuous data to centralized monitoring platforms. The study demonstrates improved detection accuracy and reduced false alarms, though computational complexity and processing power requirements remain challenges.

[3] Karne R. (2025). "Embodied AI for Security: Robotic Agents in Surveillance, Disaster Response, and Cyber-Physical Defense," Journal of Autonomous Systems and Robotics, Vol. 11, No. 1, pp. 1–12.

Karne explored the concept of embodied AI in robotic systems for surveillance and disaster response applications. The research highlights the integration of machine learning, sensor fusion and adaptive decision-making in autonomous robotic agents capable of analyzing and responding to real-world scenarios. While emphasizing improved operational efficiency and reduced human intervention, the study also discusses challenges such as computational demands, ethical considerations and system reliability.

[4] Karmakar H., Majumder R., Sarkar M., & Bhattacharjee S. (2024). "Spy–Rover: An IoT Based Surveillance Robot," International Journal of Innovative Research in Electrical, Electronics, Instrumentation and Control Engineering, Vol. 12, No. 4, pp. 101–108.

Karmakar et al. introduced the Spy-Rover, an IoT-based surveillance robot designed for remote monitoring through live video streaming and wireless control interfaces. The system integrates embedded control units, camera modules, and IoT platforms to enable real-time communication and navigation. The research emphasizes cost-effectiveness and ease of deployment, though it depends on stable internet connectivity and faces potential cybersecurity risks.

[5] Kumar S., Naveen G., & Parameshachari D. (2022). "Military Robot Design and Implementation for Wireless Communication," International Journal of Defense Technology and Engineering, Vol. 8, No. 3, pp. 55–63.

Kumar, Naveen and Parameshachari developed a military surveillance robot designed to assist defense personnel in monitoring hostile territories and hazardous environments. The system integrates wireless communication modules

for remote control and real-time video transmission, along with obstacle detection sensors for safe navigation. The modular architecture allows scalability and future enhancements. The study highlights reduced manpower requirements, enhanced soldier safety, and improved operational efficiency in high-risk zones. However, limitations such as restricted communication range and battery capacity constraints are noted. The research contributes to the advancement of practical defence robotics and embedded communication systems.

[6] Nair A., Niteesha S., Sharma G., & Nalinakshan S. (2025). "Monitoring and Leak Detection of Hazardous Gas Using Internet of Things," International Journal of Smart Safety Systems, Vol. 10, No. 2, pp. 120–128.

The system utilizes gas sensors capable of detecting harmful gases such as LPG, methane and carbon monoxide, continuously monitoring air quality in real time. When gas concentration exceeds predefined safety thresholds, the system automatically triggers alerts through IoT communication platforms, sending notifications to users via mobile applications or cloud-based dashboards. The architecture includes a microcontroller unit, calibrated gas sensors, wireless communication modules, and a cloud interface for real-time data logging and analysis. The authors emphasize early detection as a critical factor in preventing explosions, fire accidents, and health hazards caused by toxic gas exposure.

[7] Pawar B., Dharwadkar V., Deshpande A., Honawad K., & Dharmadhikari A. (2021). "An Android Based Smart Robotic Vehicle for Border Security Surveillance System," International Journal of Engineering Research and Technology, Vol. 10, No. 6, pp. 342–349.

Android-based smart robotic vehicle developed for border security surveillance applications using wireless communication technology. The robot is

equipped with a camera for live video streaming and sensors for obstacle detection and navigation, while control is achieved through an Android mobile application. The system enables remote movement control, real-time monitoring, and instant data transmission, reducing the need for continuous human patrol in sensitive areas. The study highlights cost-effectiveness, user-friendly interface, and improved operational efficiency, though communication range and environmental conditions may impact system reliability.

[8] Rao K., Maddala A., & Mangali A. (2025). "Intelligent Gesture-Based Robotic System with MEMS and RF Technology for Enhanced Motion Control using ESP32-CAM," International Journal of Smart Robotics and Automation, Vol. 14, No. 1, pp. 23–31.

The integration of ESP32-CAM enables real-time video streaming, allowing the operator to monitor the surroundings remotely. The system enhances human-robot interaction by eliminating traditional remote controllers and replacing them with intuitive gesture commands, improving usability and response time. The authors highlight its applications in defense, surveillance, and hazardous environments where direct human intervention is risky, while noting that gesture recognition accuracy and signal interference can influence system performance.

[9] Singh I., Kumar V., Aarthy T., Akilan T., & Sharma V. (2024). "Real-Time Monitoring and Surveillance in Hazardous Environments using IoT-enabled Robotics," Journal of Industrial IoT and Robotics, Vol. 7, No. 4, pp. 77–85.

An IoT-enabled robotic system for real-time monitoring in hazardous environments such as industrial plants, disaster zones, and toxic areas. The robot

integrates environmental sensors to measure temperature, gas concentration, and humidity, along with a camera module for visual surveillance. IoT connectivity enables cloud-based data transmission, real-time monitoring, and automated alert generation when safety thresholds are exceeded. The study emphasizes improved situational awareness, rapid emergency response, and reduced human exposure to dangerous conditions. However, stable network connectivity and efficient power management are critical factors influencing system reliability.

[10] Thendral R., Sanjay S., & Naveen S. (2025). "IoT-Enabled Battlefield Surveillance Robot with Metal Detection and Real-Time Alerts," International Journal of Advanced Research in Engineering and Technology, Vol. 16, No. 2, pp. 45–52.

Thendral, Sanjay and Naveen proposed an IoT-enabled battlefield surveillance robot integrated with metal detection and real-time alert systems for military applications. The robot combines obstacle detection sensors, metal detection units for identifying buried explosives or weapons, and a camera module for live video transmission. IoT-based communication ensures continuous data monitoring and instant alert notifications to remote control stations. The system enhances situational awareness, reduces direct soldier exposure in hostile zones, and improves operational efficiency. Limitations include battery life constraints and adaptability challenges in rugged terrain environments.

.

CHAPTER 3

SYSTEM DESIGN

3.1 EXISTING SYSTEM

Existing border surveillance and hazardous zone monitoring systems mainly depend on traditional security infrastructure and manual supervision. These systems include manual patrolling, fixed CCTV cameras, watchtowers, alarm systems, and standalone environmental sensors. While they provide a basic level of monitoring and protection, they are not fully integrated or mobile. In large border regions, industrial areas, and disaster-prone zones, these conventional systems face limitations in coverage, response time, and operational efficiency. The lack of unified real-time monitoring and automated response mechanisms reduces their effectiveness in critical situations. Therefore, although existing systems serve fundamental surveillance purposes, they are not sufficient for modern, dynamic, and high-risk environments.

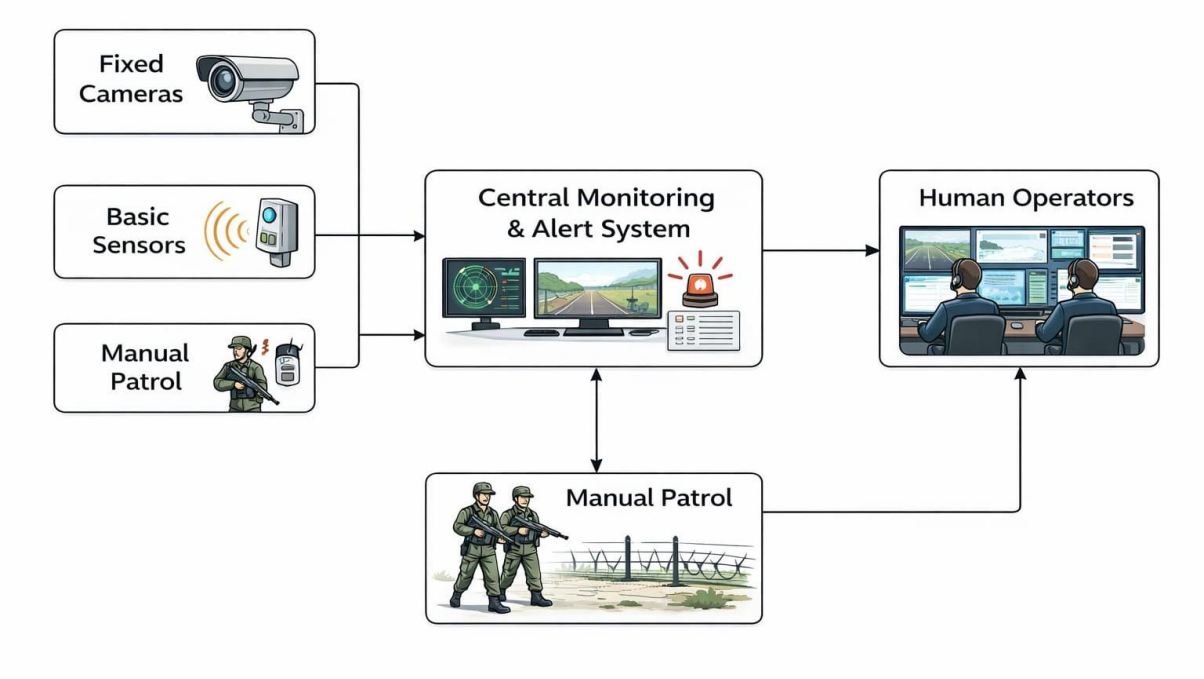


Figure 3.1: Existing System

The major drawback of existing systems is their dependence on manual supervision and fixed infrastructure. Manual patrolling exposes security personnel to risks such as harsh weather conditions, difficult terrain, and hostile threats. Fixed CCTV cameras, although effective within limited areas, cannot adjust their position to cover blind spots or moving targets. Standalone fire and gas detection systems provide alerts but do not offer visual confirmation or integrated response mechanisms.

Additionally, many systems lack real-time cloud integration, limiting remote accessibility and centralized monitoring. Advanced robotic or aerial systems, while effective, are often expensive and complex to maintain, making them less accessible for widespread deployment. The fragmented architecture of sensing, monitoring, and communication modules results in delayed response times and reduced situational awareness. Overall, the existing system provides partial surveillance and hazard detection capabilities but lacks mobility, integration, automation, and cost-effectiveness. These limitations highlight the need for a unified IoT-based robotic surveillance platform that combines sensing, communication, mobility, and real-time monitoring into a single efficient system.

3.2 PROPOSED SYSTEM

The proposed system presents an IoT-enabled embedded robotic vehicle developed for border surveillance and hazardous zone monitoring. The primary objective of the system is to reduce human involvement in dangerous environments while ensuring continuous and reliable monitoring. By integrating embedded systems, wireless communication and sensor technologies, the system provides real-time observation, intelligent threat detection and remote operational control.

The robotic platform combines mobility, environmental sensing, cloud connectivity and live video streaming into a single coordinated system. The ESP32 microcontroller serves as the central processing unit, managing sensor data acquisition, wireless communication and motor control. Its built-in Wi-Fi capability allows seamless connection to cloud platforms, enabling authorized users to monitor and control the vehicle from remote locations through a mobile or web interface.

The system is designed with a modular and scalable architecture, allowing additional sensors or advanced features to be integrated in the future. Cost-effective components and efficient power management ensure practical implementation in real-world security and hazardous monitoring applications. Overall, the proposed system enhances surveillance efficiency, improves safety and strengthens security operations.

3.2.1 Process

The working process of the proposed system begins when power is supplied to the robotic vehicle. The power supply unit regulates voltage and activates the ESP32 microcontroller along with all connected modules. During initialization, the microcontroller configures input-output pins, establishes Wi-Fi connectivity, and calibrates connected sensors.

Once initialization is complete, the system enters continuous monitoring mode. The ESP32 collects real-time data from the ultrasonic sensor, fire detection sensor, and human presence sensor. The ultrasonic sensor measures distance to detect nearby obstacles, while the fire sensor identifies abnormal temperature or flame conditions. The human presence sensor detects movement in restricted areas. Sensor readings are processed and compared with predefined threshold values. If no abnormal condition is detected, the system continues routine

surveillance and transmits status updates to the cloud platform. If an obstacle is detected within a critical range, the controller sends control signals to the L293D motor driver to stop or change direction, preventing collision.

In the case of fire detection or unauthorized human presence, the system generates instant alerts. Notifications are transmitted through the IoT interface, and relevant information is displayed on the LCD screen. Simultaneously, the mobile camera module provides live video streaming, enabling visual verification of the situation. Remote users can send control commands through the IoT platform to navigate the robotic vehicle in forward, backward, left, or right directions. The ESP32 processes these commands and activates the motors accordingly. This entire process operates in a continuous loop, ensuring uninterrupted surveillance, responsive action, and real-time monitoring.

3.3 BLOCK DIAGRAM

The following figure illustrates the block diagram of the Robotic Vehicle for Border Surveillance and Hazardous Zone Operations.

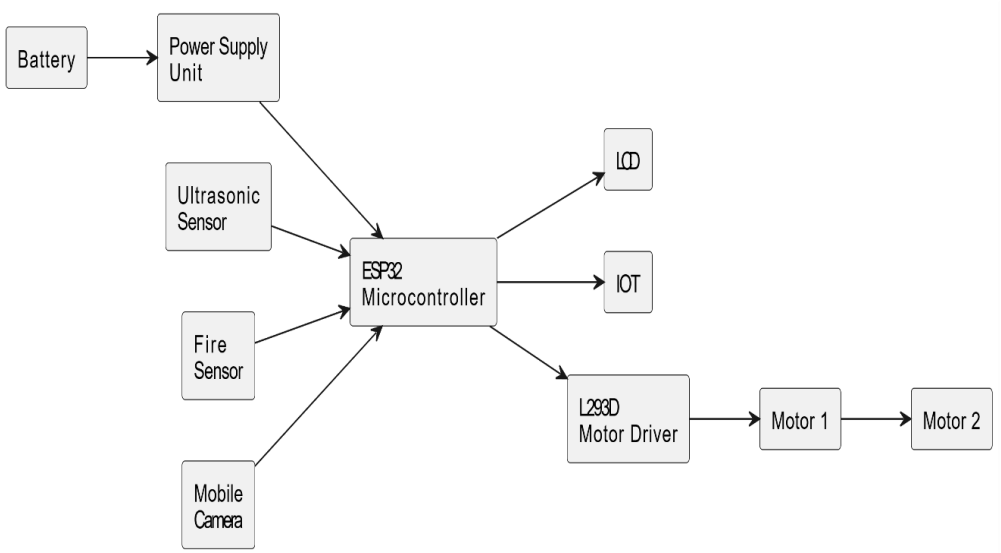


Figure 3.2: Block Diagram

3.4 EXPLAINATION

The block diagram shows the overall structure of the IoT-based surveillance robotic vehicle developed for monitoring and operation in hazardous environments. At the core of the system is the ESP32 microcontroller, which functions as the main control unit, managing and coordinating all connected components.

The system is powered by a battery, and a power supply unit is used to regulate and distribute stable voltage to ensure reliable functioning. The ESP32 receives input from multiple modules, including an ultrasonic sensor for obstacle detection and distance measurement, and a fire sensor for identifying fire or high-temperature conditions. A mobile camera is also integrated to capture real-time video, enabling remote visual monitoring of the surroundings.

All the collected data is processed by the ESP32 and transmitted through the IoT module, allowing users to monitor the system remotely via a network or cloud platform. This feature enhances safety by reducing the need for human presence in dangerous areas.

For movement, the ESP32 controls an L293D motor driver, which provides the required current to operate two DC motors. These motors enable the robotic vehicle to move in different directions, ensuring flexibility in navigation. Additionally, an LCD display is included to show real-time system information such as sensor status and alerts.

In summary, the system combines sensing, communication, and motion control into a single embedded platform. This integration makes the robotic vehicle efficient, reliable, and suitable for surveillance and hazardous zone operations.

CHAPTER 4

HARDWARE DESCRIPTION

4.1 HARDWARE REQUIREMENTS

1. ESP32 Microcontroller
2. L293D Motor Driver
3. Battery
4. Power Supply Unit
5. Ultrasonic Sensor
6. Fire Sensor
7. LCD Display
8. Dc Motor

4.1.1 ESP32 Microcontroller

The Expressive Systems ESP32 is a powerful and versatile microcontroller widely used in Internet of Things (IoT) applications. It features built-in Wi-Fi and Bluetooth connectivity, enabling seamless communication with other devices and cloud platforms. The ESP32 is powered by a dual-core Tensilica LX6 processor running up to 240 MHz, allowing it to handle multitasking and real-time control operations efficiently. The microcontroller includes 520 KB of SRAM and supports external SPI flash memory up to 16 MB, making it suitable for data logging and complex programs. It offers multiple peripherals such as programmable GPIO pins, 12-bit ADC, DAC, and communication interfaces like

SPI, I2C, and UART, enabling easy integration with various sensors and modules. These features make it ideal for embedded and robotic applications.

The ESP32 also supports advanced power-saving modes, including deep sleep, which reduces power consumption in battery-operated systems. Security features such as hardware encryption, secure boot, and flash encryption ensure safe data transmission in IoT networks. Supported by development platforms like Arduino IDE and ESP-IDF, the ESP32 is widely used in smart home systems, environmental monitoring, industrial automation, and robotics. Overall, it combines performance, connectivity, and flexibility in a compact and cost-effective solution.



Figure 4.1: ESP32 DevKit Full Pinout

4.1.1.1 ESP32 Pin Functions

- GPIO Pins: Up to 34 configurable General Purpose Input/Output pins for versatile applications. These pins can be programmed for digital input or output, making them suitable for interfacing with a variety of sensors and actuators.

- ADC: Up to 18 channels with 12-bit resolution for accurate analog signal measurement. This feature allows the ESP32 to read varying voltage levels from sensors, providing precise data for applications like environmental monitoring.
- DAC: 2 channels with 8-bit resolution for generating analog outputs. The DAC can be used for audio applications or to control devices that require variable voltage levels.
- Power Pins: V5 for 5V input, V3 for 3.3V output, and multiple GND pins. These power pins facilitate connections to external devices and ensure that the ESP32 operates within its required voltage range.
- I2C Interface: SDA and SCL pins for connecting I2C devices. The I2C protocol allows multiple devices to share the same bus, simplifying the wiring and connections needed for complex projects.
- SPI Interface: Standard pins for MISO, MOSI, SCK, and CS for SPI communication. SPI is a high-speed interface ideal for connecting fast peripherals, such as sensors and memory cards.
- UART Interface: TX and RX pins for serial communication. These pins enable the ESP32 to communicate with other devices over a serial interface, useful for debugging and data transmission.
- Touch Sensor Pins: Configurable pins for touch-sensitive input. This feature allows for the development of user interfaces that can detect touch input, enhancing interactivity in applications.
- RTC Functions: Pins for real-time clock operations and wake-up triggers. These functions are essential for applications requiring time-based operations, even during low power modes.
- Programming/Debugging: EN pin for chip enable and IO0 for boot mode selection during programming. These pins are crucial for loading new firmware and debugging the ESP32 during development.

4.1.1.2 ESP32 Specifications

- Microprocessor: Dual-core Tensilica LX6 processor with up to 240 MHz clock speed. This architecture allows for efficient handling of multiple tasks simultaneously, making it suitable for demanding applications.
- Memory: 520 KB of SRAM and support for up to 16 MB of external SPI flash memory. The ample memory capacity enables the execution of complex algorithms and the storage of extensive data.
- Wireless Connectivity: Integrated Wi-Fi (IEEE 802.11 b/g/n) and Bluetooth (Classic and BLE). This dual connectivity enhances the flexibility of the ESP32 in IoT applications, allowing seamless communication with various devices.
- GPIO Pins: Up to 34 General Purpose Input/Output pins for various applications. These pins can serve multiple functions, such as analog input, PWM output, or digital signaling, making the ESP32 adaptable to different project needs.
- Analog Features: 12-bit ADC with up to 18 channels and 2 channels of 8-bit DAC. The high-resolution analog features are ideal for applications requiring precise signal measurement and generation.
- Power Supply: Operates within a voltage range of 3.0V to 3.6V. This range allows the ESP32 to be powered by various sources, including batteries and regulated power supplies, enhancing its usability in portable applications.
- Power Management: Multiple power-saving modes, including deep sleep. These modes significantly reduce power consumption, making the ESP32 suitable for battery-operated devices and energy-efficient applications.
- Operating Temperature: Functional range from -40°C to 85°C. This wide temperature range ensures reliable performance in diverse environmental conditions, from industrial applications to outdoor deployments.

- Security Features: Hardware encryption (AES, SHA-2), secure boot, and flash encryption. These security measures help protect sensitive data and ensure the integrity of the device, which is critical in IoT applications.
- Development Support: Compatible with Arduino IDE, Expressive ESP-IDF, and other frameworks. The extensive development support, along with a rich set of libraries and resources, simplifies the programming process and accelerates project development.

4.1.2 L293D Motor Driver

The L293D motor driver IC (integrated circuit) used to control the direction and speed of DC motors. It's commonly used in robotics, automation, and other projects where precise motor control is required.

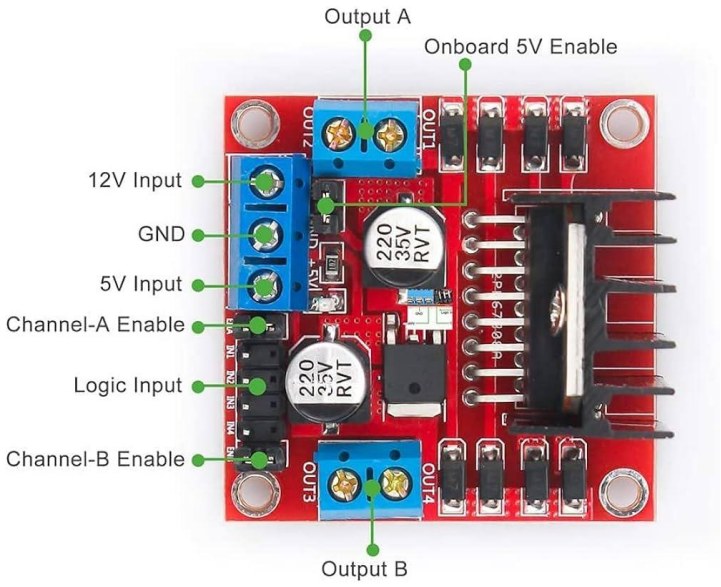


Figure 4.2: L293D Motor Driver Module

4.1.2.1 Functionality

The L293D is a quadruple high-current half-H driver designed to drive inductive loads such as DC motors, stepper motors, and solenoids. It can drive up to two DC motors bidirectionally (forward and reverse) with individual control of motor speed and direction.

4.1.2.2 Features

- Dual H-Bridge: Contains two H-bridges (each consisting of four transistors) to control two motors independently.
- Current Capability: Can handle a continuous current of up to 600mA per channel and peak currents of up to 1.2A per channel (with proper heat sinking).
- Built-in Diodes: Includes built-in flyback diodes (freewheeling diodes) for protection against voltage spikes generated when driving inductive loads.

4.1.2.3 Pin Configuration

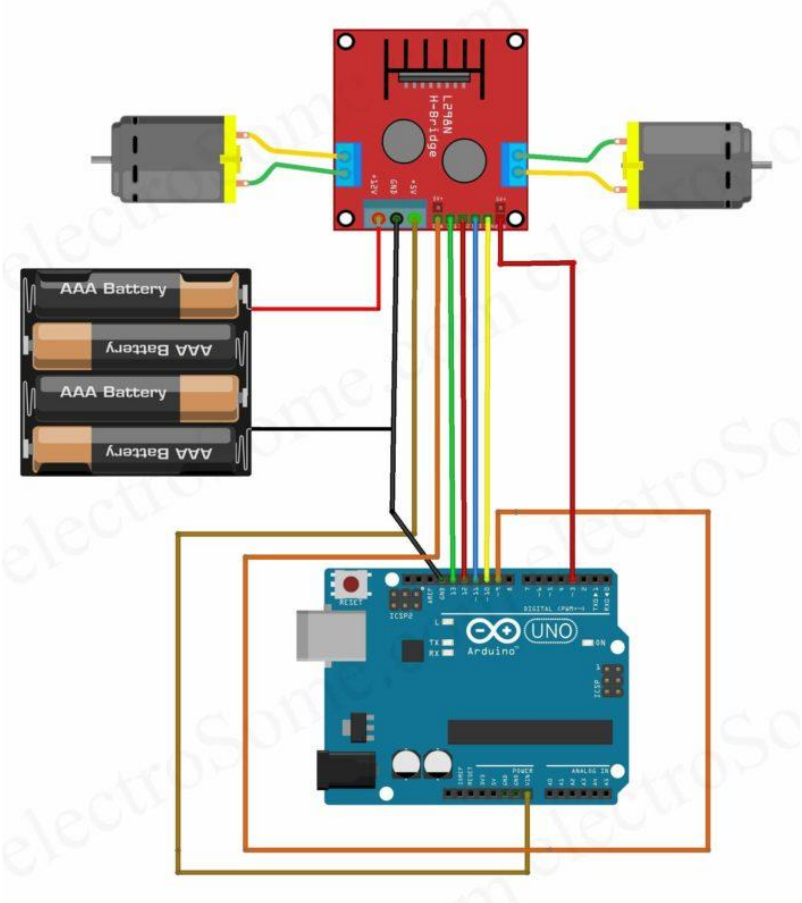


Figure 4.3: Arduino Uno Interfacing with L298N Motor Driver Module for Dual DC Motor Control

- VCC1 (Pin 16): Power supply pin for internal logic (typically connected to +5V).
- VCC2 (Pin 8): Power supply pin for motor driving (typically connected to the motor supply voltage, e.g., +12V).
- Enable Pins (1, 9): Enable pins for each H-bridge; active HIGH enables the corresponding outputs.
- Input Pins (2, 7, 10, 15): Input pins for controlling motor direction and speed (through PWM for speed control).
- Output Pins (3, 6, 11, 14): Output pins connected to the motor terminals.
- Ground Pins (4, 5, 12, 13): Ground connections.

4.1.2.4 Operations

- To control a DC motor using the L293D, you provide logic signals to the input pins (2, 7, 10, 15) to set the direction (forward or reverse) and use PWM signals for speed control (if required).
- The Enable pins (1, 9) are typically connected to digital output pins of a microcontroller to enable or disable motor operation.

4.1.3 Battery

4.1.3.1 Lithium-Ion Batteries

Lithium-ion (Li-ion) batteries have become one of the most widely used energy storage technologies in the modern world. Their high energy density, long cycle life, and relatively low self-discharge rate make them ideal for a wide range of applications, from portable electronics to electric vehicles (EVs) and

renewable energy storage systems. Since their commercialization in the early 1990s, Li-ion batteries have revolutionized the way we store and use energy, enabling advancements in technology and contributing to the global shift toward cleaner energy solutions. This report provides a comprehensive description of lithium-ion batteries, covering their working principles, components, advantages, disadvantages, and applications. Additionally, it explores the challenges and future prospects of this technology.



Figure 4.4: 18650 lithium-ion battery

4.1.3.2 Working Principle

Lithium-ion batteries operate based on the movement of lithium ions between the anode and cathode during charging and discharging cycles. The fundamental principle involves electrochemical reactions that store and release energy.

Charging Process: During charging, lithium ions move from the cathode to the anode through the electrolyte. The anode, typically made of graphite, stores the lithium ions in its layered structure. Electrons flow through the external circuit, providing energy to the battery.

Discharging Process: During discharging, lithium ions move back from the anode to the cathode, releasing energy in the form of electrical current. The electrons flow through the external circuit, powering the connected device.

The chemical reactions at the electrodes are as follows:

Anode (Graphite):

$$Li\ x\ C\ 6 \rightarrow C\ 6 + xLi + + xe -$$

Cathode (Lithium Cobalt Oxide, $LiCoO_2$):

$$Li\ 1-x\ CoO\ 2 + xLi + + xe - \rightarrow LiCoO\ 2$$

The electrolyte, typically a lithium salt in an organic solvent, facilitates the movement of lithium ions while preventing the flow of electrons, ensuring efficient energy transfer.

4.1.4 Power Supply Unit

The 7812 and 7805 voltage regulators are commonly used components to provide stable DC voltage outputs of +12V and +5V, respectively, from a higher input voltage source.

4.1.4.1 7812 Voltage Regulator

- Input Voltage: Typically requires an input voltage slightly higher than 12V (usually around 14-16V) to regulate effectively.
- Output Voltage: Provides a stable +12V DC output.
- C1000/25: This likely refers to a capacitor with a capacitance of 1000µF and a voltage rating of 25V. This capacitor is typically placed on the input side (between input and ground) to stabilize the input voltage, reducing noise and providing a reservoir of charge to handle transient spikes.

- C10/63: This could refer to a capacitor with a capacitance of 10µF and a voltage rating of 63V. This capacitor is usually placed on the output side (between output and ground) to stabilize the output voltage, filtering out any remaining noise and improving regulation.
- Resistor: A resistor isn't typically used directly with the 7812 regulator in the same way as capacitors are, but it can be part of the circuit design for specific applications, such as in voltage dividers or as part of a feedback loop for stability.

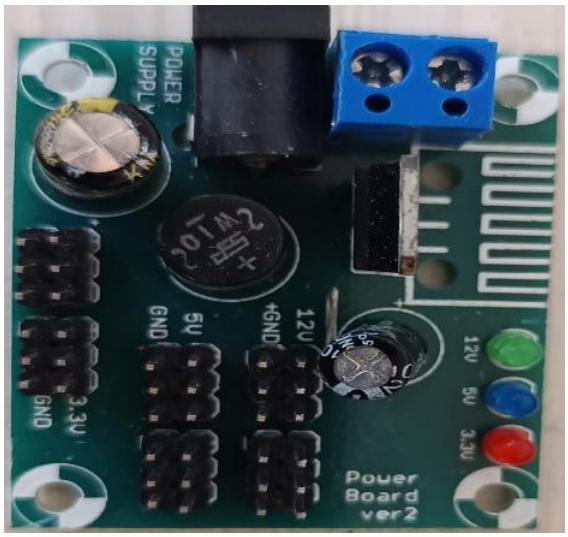


Figure 4.5: MB102 Breadboard Power Supply Module (Power Board v2)

4.1.4.2 7805 Voltage Regulator

- Input Voltage: Requires an input voltage typically around 7-25V (ideal 7-20V) to regulate effectively.
- Output Voltage: Provides a stable +5V DC output.
- Capacitors:
- C1000/25: As with the 7812, this capacitor stabilizes the input voltage to the regulator.

- C10/63: This capacitor stabilizes the output voltage of the regulator.
- Resistor: Similar to the 7812, resistors are not directly part of the typical configuration but can be used in specific applications.

4.1.4.3 Circuit Considerations

- Decoupling Capacitors: These capacitors (like C1000/25 and C10/63) are crucial for filtering out noise and stabilizing the voltage levels, ensuring reliable operation of your circuit.
- Heat Dissipation: Both regulators can generate heat, especially when dropping significant voltage. Adequate heat sinking may be necessary depending on the current drawn and the input-output voltage differential.
- Current Requirements: Ensure that the regulators can supply enough current for your application. If higher currents are required, additional heat sinking and possibly parallel regulators may be needed.
- In summary, the 7812 and 7805 voltage regulators, along with capacitors like C1000/25 and C10/63, form a basic yet effective setup for providing stable +12V and +5V outputs in electronic circuits, suitable for a wide range of applications from powering microcontrollers to analog circuits.

4.1.5 Ultrasonic Sensor

An ultrasonic sensor is an electronic device that measures the distance to an object by using high-frequency sound waves beyond the range of human hearing. It operates on the principle of echo reflection, where the sensor emits ultrasonic pulses and detects the sound waves that bounce back after hitting an object. By calculating the time taken for the echo to return and using the known speed of sound in air, the sensor determines the distance between itself and the object. Ultrasonic sensors are widely used in robotics and embedded systems for obstacle

detection, distance measurement, and collision avoidance due to their accuracy, low cost, and ease of interfacing with microcontrollers.

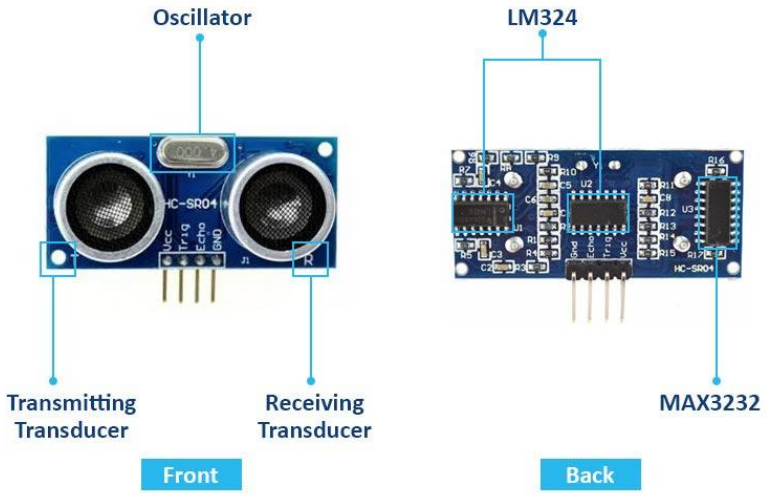


Figure 4.6: HC-SR04 Ultrasonic Sensor (Front and Back View)

4.1.5.1 Working Principle of Ultrasonic Sensor

An ultrasonic sensor works by emitting high-frequency sound waves, typically beyond the range of human hearing, and measuring the time it takes for these waves to travel to an object and bounce back. The sensor consists of a transmitter that sends out an ultrasonic pulse and a receiver that detects the reflected sound waves. When the sensor emits the sound wave, it travels through the air until it encounters an object, which reflects the wave back to the sensor. The sensor measures the time it takes for the sound to travel to the object and return. Using the speed of sound in air, the sensor calculates the distance to the object. The formula used is:

$$\text{Distance} = \frac{\text{Speed of Sound} \times \text{Time}}{2}$$

The factor of 2 accounts for the round trip of the sound wave. By processing the time and the known speed of sound, the sensor can determine the precise

distance between itself and the object. This non-contact measurement method makes ultrasonic sensors valuable in a wide range of applications, such as obstacle detection, distance measurement, and level sensing.

4.1.5.2 Pin Configuration

A common ultrasonic sensor is the HC-SR04. Below is its typical pin configuration:

VCC	Power supply pin (typically +5V).
Trig (Trigger):	Sends a 10 µs HIGH pulse to start the ultrasonic wave.
Echo	Outputs a HIGH signal with duration proportional to the return time of the wave.
GND	Ground pin for the sensor.

4.1.6 Fire Sensor

A flame sensor is a device used to detect the presence of a flame or fire. It is commonly used in safety applications, such as fire detection systems, industrial burners, and gas-powered appliances. It works by sensing the infrared or ultraviolet light emitted by a flame and generating a signal for immediate alert or control action.

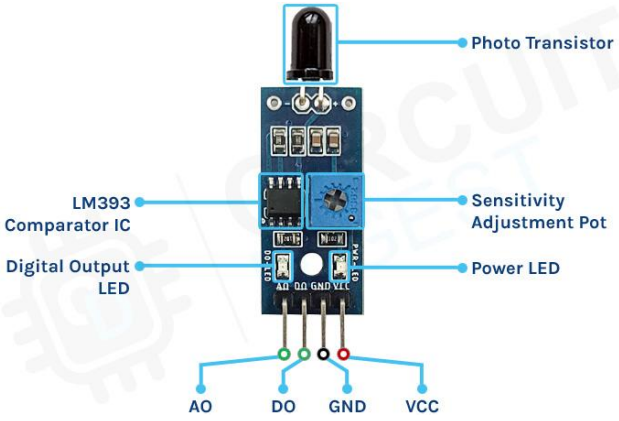


Figure 4.7: Infrared (IR) Flame Sensor Module

4.1.6.1 Key Features

- Detection Capability: Detects the presence of flame or fire based on the light emitted by the flame, typically in the infrared (IR) or ultraviolet (UV) spectrum.
- Fast Response Time: Quickly detects the presence of a flame, allowing for rapid response to potential fire hazards.
- Sensitivity: High sensitivity to flames, even at a distance, making them suitable for various applications.

4.1.6.2 Pin Configuration

VCC	5V
GND	Ground on the microcontroller
DOUT	To read the digital output signal
AOUT	Intensity measurement

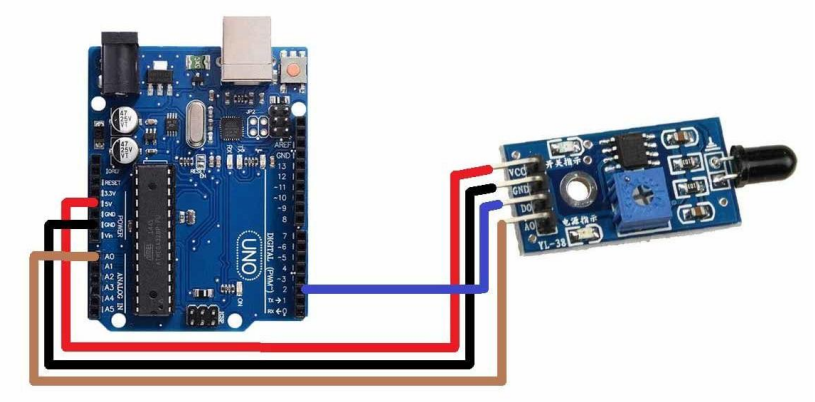


Figure 4.8: IR Flame Sensor module with an Arduino Uno

4.1.7 LCD Display

A liquid crystal display (LCD) is a flat panel display, electronic visual display, or video display that uses the light modulating properties of liquid crystals. Liquid crystals do not emit light directly. LCDs are available to display arbitrary images (as in a general-purpose computer display) or fixed images which can be displayed or hidden, such as preset words, digits, and 7-segment displays as in a digital clock. They use the same basic technology, except that arbitrary images are made up of a large number of small pixels, while other displays have larger elements. An LCD is a small low cost display. It is easy to interface with a micro-controller because of an embedded controller (the black blob on the back of the board). This controller is standard across many displays (HD 44780) which means many micro-controllers (including the Arduino) have libraries that make displaying messages as easy as a single line of code.

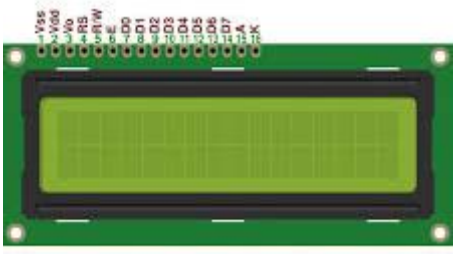


Figure 4.9: LCD display unit

LCDs are used in a wide range of applications including computer monitors, televisions, instrument panels, aircraft cockpit displays, and signage. They are common in consumer devices such as video players, gaming devices, clocks, watches, calculators and telephones, and have replaced cathode ray tube (CRT) displays in most applications. They are available in a wider range of screen sizes than CRT and plasma displays, and since they do not use phosphors, they do not suffer image burn-in. LCDs are, however, susceptible to image persistence.

4.1.7.1 16X2 LCD Specifications

- Display Format: 16 characters per line, 2 lines total.
- Character Size: 5x8 pixels for standard characters.
- Dimensions: Approximately 80mm x 36mm x 13mm.
- Interface: Parallel (4-bit or 8-bit mode).
- Supply Voltage: Typically 5V DC.
- Current Consumption: Around 1.5 mA at 5V.
- Backlight: LED backlight (3.3V to 5V).
- Temperature Range: 0°C to 70°C operating, -20°C to 80°C storage.
- Response Time: Under 10 ms.
- Mounting: PCB or breadboard compatible.
- Character Set: Standard ASCII with custom character support.

The I2C connection for an LCD display enables efficient and simplified interfacing with microcontrollers, like the ESP32, by requiring only two data lines: SDA (data) and SCL (clock). This setup significantly reduces the number of pins needed, compared to traditional parallel connections, allowing other I/O

pins on the microcontroller to remain free for additional sensors or peripherals. The I2C interface is controlled by a small I2C module on the back of the LCD, typically using a chip like the PCF8574, which converts the I2C signals into parallel signals the LCD can interpret. Each device on the I2C bus has a unique address (e.g., 0x27 for many LCD modules), letting the microcontroller communicate with specific devices even if there are multiple I2C components on the same bus. The power (VCC) and ground (GND) lines complete the connections, and the data transfer is synchronous, meaning the microcontroller (master) sends commands in sync with the LCD (slave). Commands like positioning the cursor, clearing the display, or updating text are all handled through simple I2C libraries that streamline programming. This makes I2C ideal for projects that require organized wiring and efficient communication, especially in setups where pin limitations and space are considerations.

4.1.8 Dc Motor

Geared DC motors can be defined as an extension of DC motor which already had its Insight details demystified here. A geared DC Motor has a gear assembly attached to the motor. The speed of motor is counted in terms of rotations of the shaft per minute and is termed as RPM .The gear assembly helps in increasing the torque and reducing the speed. Using the correct combination of gears in a gear motor, its speed can be reduced to any desirable figure. This concept where gears reduce the speed of the vehicle but increase its torque is known as gear reduction. This Insight will explore all the minor and major details that make the gear head and hence the working of geared DC motor.

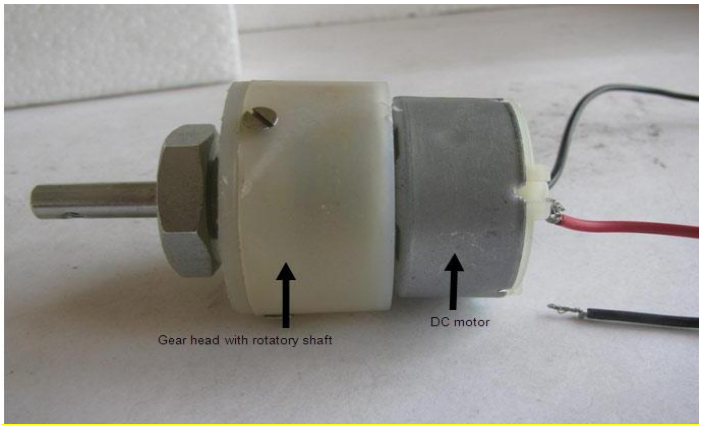


Figure 4.10: Dc Motor

The outer body of the gear head is made of high density plastic but it is quite easy to open as only screws are used to attach the outer and the inner structure. The major reason behind this could be to lubricate gear head from time to time.The plastic body has a threading through which nut can be easily mounted and vice versa from the gear head

4.1.8.1 Working of Geared Dc Motor

The DC motor works over a fair range of voltage. The higher the input voltage more is the RPM (rotations per minute) of the motor. For example, if the motor works in the range of 6-12V, it will have the least RPM at 6V and maximum at 12 V.

In terms of voltage, we can put the equation as:

RPM= K1 * V, where,

K1= induced voltage constant

V=voltage applied

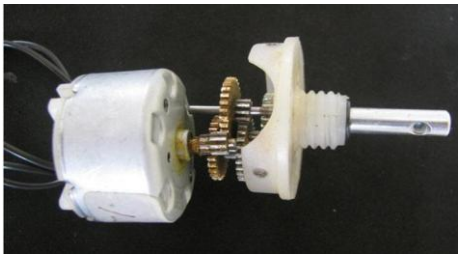


Figure 4.11: DC Geared Motor

The working of the gears is very interesting to know. It can be explained by the principle of conservation of angular momentum. The gear having smaller radius will cover more RPM than the one with larger radius. However, the larger gear will give more torque to the smaller gear than vice versa. The comparison of angular velocity between input gear (the one that transfers energy) to output gear gives the gear ratio. When multiple gears are connected together, conservation of energy is also followed. The direction in which the other gear rotates is always the opposite of the gear adjacent to it. In any DC motor, RPM and torque are inversely proportional. Hence the gear having more torque will provide a lesser RPM and converse. In a geared DC motor, the concept of pulse width modulation is applied.

In a geared DC motor, the gear connecting the motor and the gear head is quite small, hence it transfers more speed to the larger teeth part of the gear head and makes it rotate. The larger part of the gear further turns the smaller duplex part. The small duplex part receives the torque but not the speed from its predecessor which it transfers to larger part of other gear and so on. The third gear's duplex part has more teeth than others and hence it transfers more torque to the gear that is connected to the shaft.

CHAPTER 5

SOFTWARE DESCRIPTION

5.1 SOFTWARE REQUIREMENTS

1. **PROGRAMMING LANGUAGE:**

   Embedded C

2. **COMPILER :**

   Arduino IDE

3. **MOBIL APP :**

   IoT module

5.1.1 Embedded C

Embedded C serves as the functional extension of the standard C programming language, specifically optimized for microcontrollers and hardware-level operations. While it retains the syntax of C, it introduces hardware-specific features necessary for managing memory-constrained systems like the Arduino Uno.

- Hardware Interaction: Unlike desktop software, Embedded C allows for direct manipulation of hardware registers, enabling the program to control individual pins for reading sensors or driving motors.
- Resource Efficiency: It is designed to operate within the limited RAM and Flash memory of a microcontroller, ensuring that the fire-detection logic executes with minimal latency.
- Infinite Execution: Programs are structured around a continuous execution loop, allowing the system to perpetually scan for flame signals and respond instantly to environmental changes.

- Interrupt Handling: It supports interrupt service routines, which allow the microcontroller to pause regular tasks to handle high-priority events, such as an emergency stop signal.

5.1.2 Arduino IDE

The Arduino integrated development environment (IDE) is a cross-platform application (for Windows, macOS, Linux) that is written in the programming language Java. It is used to write and upload programs to Arduino board. The source code for the IDE is released under the GNU General Public License, version 2. The Arduino IDE supports the languages C and C++ using special rules of code structuring. The Arduino IDE supplies a software library from the Wiring project, which provides many common input and output procedures. User-written code only requires two basic functions, for starting the sketch and the main program loop, that are compiled and linked with a program stub main() into an executable cyclic executive program with the GNU tool chain, also included with the IDE distribution. The Arduino IDE employs the program argued to convert the executable code into a text file in hexadecimal encoding that is loaded into the Arduino board by a loader program in the board's firmware.

Arduino is an open-source electronics platform based on easy-to-use hardware and software. Arduino boards are able to read inputs - light on a sensor, a finger on a button, or a Twitter message - and turn it into an output - activating a motor, turning on an LED, publishing something online. You can tell your board what to do by sending a set of instructions to the microcontroller on the board

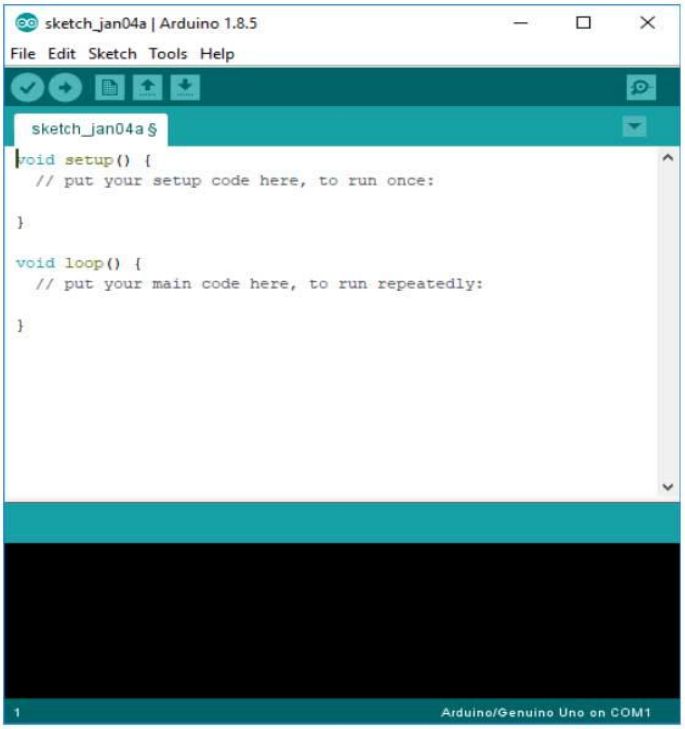


Figure 5.1: Embedded C and IoT Module

The Arduino IDE is incredibly minimalistic, yet it provides a near-complete environment for most Arduino-based projects. The top menu bar has the standard options, including “File” (new, load save, etc.), “Edit” (font, copy, paste, etc.), “Sketch” (for compiling and programming), “Tools” (useful options for testing projects), and “Help”.

The middle section of the IDE is a simple text editor that where you can enter the program code. The bottom section of the IDE is dedicated to an output window that is used to see the status of the compilation, how much memory has been used, any errors that were found in the program, and various other useful messages.

Projects made using the Arduino are called sketches, and such sketches are usually written in a cut-down version of C++ (a number of C++ features are not included). Because programming a microcontroller is somewhat different from programming a computer, there are a number of device-specific libraries (e.g.,

changing pin modes, output data on pins, reading analog values, and timers). This sometimes confuses users who think Arduino is programmed in an “Arduino language.” However, the Arduino is, in fact, programmed in C++. It just uses unique libraries for the device.

The Arduino Integrated Development Environment - or Arduino Software (IDE) - contains a text editor for writing code, a message area, a text console, a toolbar with buttons for common functions and a series of menus. It connects to the Arduino and Genuine hardware to upload programs and communicate with them.

Programs written using Arduino Software (IDE) are called sketches. These sketches are written in the text editor and are saved with the file extension .ino. The editor has features for cutting/pasting and for searching/replacing text.

The message area gives feedback while saving and exporting and also displays errors. The console displays text output by the Arduino Software (IDE), including complete error messages and other information. The bottom right hand corner of the window displays the configured board and serial port. The toolbar buttons allow you to verify and upload programs, create, open, and save sketches, and open the serial monitor.

5.1.3 IOT Module

The IoT Module (such as the ESP8266 or similar) provides the system with "cloud intelligence," enabling it to transcend local operation and become part of a larger interconnected network.

- Wireless Data Gateway: This module establishes a transparent bridge between the Arduino’s serial data and the internet, allowing for the wireless transmission of flame alerts.

- Mobile App Communication: It facilitates bi-directional communication, where sensor data is pushed to a Mobile App for monitoring, and user commands are pulled from the app to control the robot.
- Protocol Management: The module handles complex networking protocols (like MQTT or HTTP) internally, allowing the main Arduino processor to focus entirely on fire detection and navigation.
- Remote Telemetry: Beyond emergency alerts, the IoT module can log historical data to a cloud database, providing insights into the frequency and location of detected hotspots over time.

5.2 LIBRARIES

Libraries offer additional functionality for use in sketches, such as working with hardware or processing data. To use a library in a sketch, it can be selected from the Sketch > Import Library menu. This action will add one or more #include statements at the beginning of the sketch and compile the library along with the program. Since libraries are uploaded to the board together with the sketch, they occupy additional memory space. If a library is no longer required, its #include statements can be removed from the code.

A list of available libraries is provided in the reference section. Some libraries are included with the Arduino software by default, while others can be downloaded from a various source or installed using the Library Manager. From version 1.0.5 of the IDE, libraries can also be imported from a zip file and used directly in a sketch.

5.3 CONNECTING THE ARDUINO

Connecting an Arduino board to your PC is quite simple. On Windows:

1. Connect the USB cable - one end to the PC, and one end to the Arduino board.

2. When prompted, choose "Browse my computer for drivers" and select the folder where the Arduino IDE was extracted.

3. You may receive a warning that the board is not a Microsoft-certified device - select “Install anyway.”

4. Your board should now be ready for programmed.

When programming your Arduino board, it is important to identify the COM port being used on your PC. On Windows, go to Start -> Devices and Printers, and locate the Arduino device. The assigned COM port will be displayed it.

Alternatively, a notification message in the lower-left corner of the screen after successful connection usually indicates the COM port being used.

5.4 PREPARING THE BOARD

Before uploading any code to your Arduino board, you must first open the IDE. Double click the Arduino .exe file that you downloaded earlier. A blank program, also called a "sketch," will open.

The Blink example is the simplest way to test an Arduino board. In the Arduino window, it can be accessed through File->Examples->Basics->Blink.

Before uploading the code to the board, two essential steps must be completed:

1. Choose your Arduino board from the list under Tools->Board. The commonly used board RBE 1001, 2001, and 2002 is the Arduino Mega 2560, so select the "Arduino Mega 2560 or Mega ADK" option from the dropdown menu.
2. Select the communication port (COM port) by navigating to Tools->Serial Port.

If you have noted the COM port used by your Arduino board, it should be in the dropdown menu. If not, the board may not completed installation or might need to be reconnected.

5.5 LOADING CODE

The upper left of the Arduino window contains two buttons: A checkmark to Verify the code and a right-facing arrow to upload it. Click the upload button to compile and upload the Blink example to your Arduino board.

The black bar at the bottom of the Arduino window displays messages indicating whether the code upload was successful or not. A "Completed Successfully" will appear once the upload process is finished. If an error message appears, ensure that the correct board and COM port are selected in the Tools menu, and also verify the hardware connections.

If the upload is successful, the LED on the board will blink on and off every second. Most Arduino boards have an inbuilt LED connected to pin 13.

It is important not to use pins 0 or 1 while uploading code, as they may interfere with communication. It is generally recommended to avoid using these pins.

Arduino programs are transferred to the controller through a serial port. Older versions use an FTDI chip to handle USB communication, while newer models include either a small AVR acting like an FTDI chip or a built-in USB-to-serial interface within the microcontroller.

CHAPTER 6

IMPLEMENTATION & ADVANTAGES

6.1 IMPLENTATION

Residential Homes: The system can be installed in homes to protect windows from unauthorized entry, providing an affordable and effective home security solution.

Apartments and Flats: For apartment dwellers, the system offers an easy-to-install solution to safeguard windows and deter potential intruders.

Vacation Homes: The system can be used in vacation homes or rental properties to provide security when the property is unoccupied, preventing break-ins and theft.

Small Businesses: Small business owners can install the system to protect their storefronts or windows, ensuring a cost-effective way to enhance security after hours.

Windows with High Security Risks: The system is ideal for windows that are more vulnerable to break-ins, such as ground-floor or poorly secured windows in residential or commercial buildings.

Smart Homes: The system can be integrated into smart home setups, allowing for additional remote monitoring and integration with other IoT-based security systems.

6.2 ADVANTAGES

Cost-Effective: The system offers an affordable solution for enhancing home security, making it accessible for homeowners without the high costs associated with traditional alarm systems.

Easy Installation: The Arduino-based system is simple to install, allowing homeowners to set it up with minimal technical knowledge or experience.

Real-Time Alerts: The system provides immediate alerts through an audible alarm and LED indicator, ensuring quick responses to potential intrusions and enhancing overall security.

Customizable: The system is flexible and can be easily customized to fit different window types, residential setups, or specific security needs, offering versatility.

Low Maintenance: With its straightforward design and use of basic components, the system is easy to maintain, requiring minimal intervention or repairs.

Energy Efficient: The system is power-efficient, as it only activates when motion is detected, reducing unnecessary energy consumption.

CHAPTER 7

SOFTWARE OUTPUT

7.1 ARDUINO IOT CLOUD

Figure 7.1: Arduino Iot Cloud

Arduino IoT Cloud is a robust and versatile platform that enables users to develop, manage, and deploy Internet of Things (IoT) applications with ease. It is designed to integrate seamlessly with Arduino boards and other compatible hardware, allowing makers, hobbyists, and developers to create connected devices and control them remotely through a straightforward, intuitive interface.

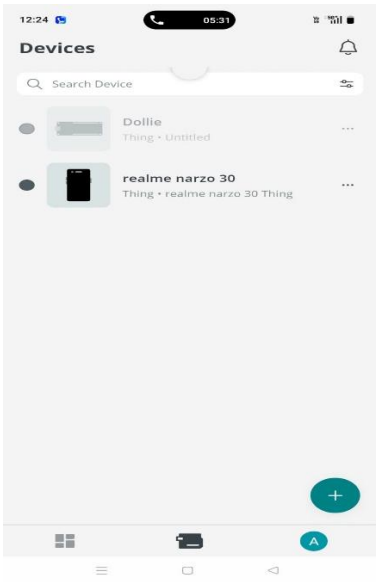


Figure 7.2: Login Device Page

By leveraging this platform, users can build projects that gather sensor data, trigger actions, and visualize information in real time from anywhere in the world. One of the defining aspects of Arduino IoT Cloud is its user-friendly design, which lowers the barrier to entry for IoT development. It provides a visual interface where users can create things, define variables, and link devices without extensive coding knowledge. This no-code or low-code approach makes the platform accessible to beginners while still being powerful enough for advanced users who want to dive deep into custom development. The platform supports a wide range of devices, including Arduino boards like the MKR series, Nano 33 IoT, and even third-party boards compatible with the Arduino ecosystem. Users can program these boards directly from the cloud, eliminating the need for a local development environment.

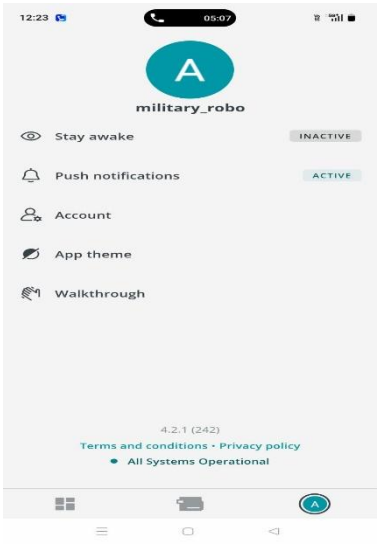


Figure 7.3: Project User Id

The Arduino Web Editor allows coding, compiling, and uploading sketches directly through a web browser, streamlining the development process and making it more flexible. With Arduino IoT Cloud, users can easily create dashboards to monitor and control their devices. These dashboards support various widgets, such as buttons, sliders, gauges, and graphs, to visualize data

and interact with connected devices. The drag-and-drop interface for building dashboards means that users can quickly design custom control panels without writing complex front-end code. Security is a core consideration in Arduino IoT Cloud, with features like secure device authentication and data encryption. Devices are linked to the cloud using secure credentials, and communication is encrypted to protect sensitive information. This ensures that users can trust their connected devices to operate safely, even when accessed remotely over public networks. Arduino IoT Cloud integrates with popular services and platforms, allowing users to build complex systems that connect with other web services and APIs.

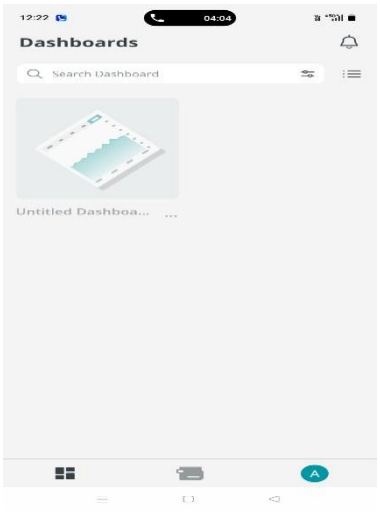


Figure 7.4: Dashboard

For example, users can set up automation rules to trigger actions based on specific conditions, such as turning on lights when a sensor detects motion or sending an alert when temperature thresholds are exceeded. Integrations with platforms like Amazon Alexa and Google Assistant also enable voice-controlled interactions, adding another layer of convenience. The platform includes extensive documentation, tutorials, and an active community that makes learning and troubleshooting easier. Arduino's open-source ethos means that users can access and modify source code, contribute to community projects, and share their

own creations. This collaborative environment encourages innovation and continuous learning, as users can draw inspiration from one another's work and build on shared knowledge. One of the key strengths of Arduino IoT Cloud is its scalability. Users can start with simple projects, like controlling an LED or reading data from a temperature sensor, and gradually progress to more complex applications, such as home automation systems, environmental monitoring, or industrial IoT solutions.

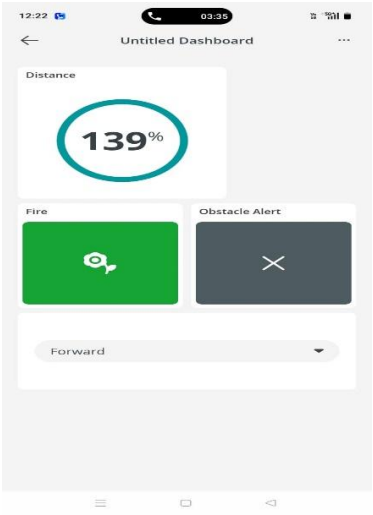


Figure 7.5: Control and Detection

The platform's flexible structure and ability to manage multiple devices make it suitable for both small personal projects and larger deployments. The real-time data capabilities of Arduino IoT Cloud are another highlight. Users can monitor live sensor readings, track historical data, and visualize trends over time. This is invaluable for applications that rely on continuous monitoring, such as agriculture, healthcare, and smart cities. By understanding data patterns, users can make informed decisions and optimize their systems for better performance. Setting up an Arduino IoT Cloud project is straightforward. Users need to create an account, add a device, and define variables that represent the data points they want to monitor or control. The platform automatically generates code snippets to handle cloud connectivity, significantly reducing development time. Once the

device is online, users can immediately start interacting with it through their custom dashboards or mobile apps. Arduino IoT Cloud also supports mobile access, enabling users to control and monitor their projects from smartphones or tablets. This adds flexibility, as users can receive push notifications, adjust settings, or check system status on the go. The mobile-friendly interface ensures a seamless experience across devices, making remote management convenient and effective. In educational settings, Arduino IoT Cloud serves as a powerful learning tool. It allows students to experiment with IoT concepts, understand real-world applications, and develop problem-solving skills. Teachers can design interactive lessons where students build connected devices, analyze data, and create their own IoT ecosystems.

The platform's simplicity makes it an excellent choice for classrooms, makerspaces, and workshops. The Arduino IoT Cloud subscription model offers a free tier for hobbyists and small projects, along with paid plans for users who need more devices, extended data retention, or additional features. This flexible pricing structure ensures that users only pay for what they need, making the platform accessible to a broad audience. By providing an all-in-one solution for IoT development, Arduino IoT Cloud empowers users to bring their ideas to life. It bridges the gap between hardware and software, making it easier than ever to create connected systems that interact with the physical world. Whether building a smart home, developing a wearable device, or prototyping an industrial solution, Arduino IoT Cloud offers the tools and support needed to succeed in the exciting world of IoT. Arduino IoT Cloud embodies the spirit of innovation and collaboration that defines the Arduino ecosystem. It fosters creativity, encourages experimentation, and helps users turn concepts into reality. With its blend of simplicity, power, and community support, it stands out as an invaluable platform for anyone looking to explore the endless possibilities of the Internet of Things.

CHAPTER 8

RESULT & DISCUSSION

8.1 RESULT

The developed IoT-based surveillance robotic vehicle was successfully implemented and tested to evaluate its performance in controlled monitoring scenarios. The system demonstrated reliable integration of sensing, communication, and mobility functions within a compact embedded architecture. All hardware modules operated cohesively under the supervision of the ESP32 microcontroller, confirming stable coordination between input, processing, and output components.

During testing, the power supply unit efficiently regulated battery output, ensuring uninterrupted functionality of the controller, sensors, display, and motor driver. The robotic platform exhibited smooth multidirectional movement through the L293D motor driver, enabling accurate forward, backward, and turning operations. Navigation performance remained stable across flat and moderately uneven surfaces, validating effective motor control and response handling.

The ultrasonic sensor accurately measured obstacle distance within its specified range, allowing the system to detect nearby objects and prevent potential collisions. Real-time distance readings were processed instantly, demonstrating consistent responsiveness. The fire detection sensor effectively identified flame presence and abnormal heat conditions under simulated testing environments. Upon detection, the system generated immediate alerts and transmitted notifications through the IoT interface, confirming proper threshold evaluation and event handling.

Wireless communication through the ESP32's built-in Wi-Fi module functioned reliably, enabling seamless cloud connectivity. Sensor data was transmitted to the remote monitoring interface without significant delay, ensuring continuous observation. The IoT platform allowed remote users to control robotic movement in real time, verifying bidirectional communication between the operator and the vehicle. Command execution latency remained minimal, supporting efficient remote navigation.

The integrated mobile camera provided stable live video streaming, delivering clear visual feedback during operation. Visual monitoring enhanced situational awareness by allowing users to verify sensor-based alerts and assess surroundings remotely. The LCD display successfully presented local system information, including sensor status and operational updates, offering on-site diagnostic visibility.

Overall system performance indicated strong synchronization between sensing modules, control logic, and communication protocols. The prototype maintained operational stability over extended test durations without overheating or unexpected interruptions. Energy consumption remained within acceptable limits for battery-powered deployment, supporting practical field usage.

The results confirm that the proposed embedded robotic system effectively achieves real-time surveillance, hazard detection, and remote mobility control within a unified framework. The integration of IoT connectivity with sensor-driven monitoring provides an efficient and responsive security solution. The successful implementation validates the feasibility of using cost-effective embedded components to create a scalable and reliable monitoring platform suitable for border security and hazardous environment applications.

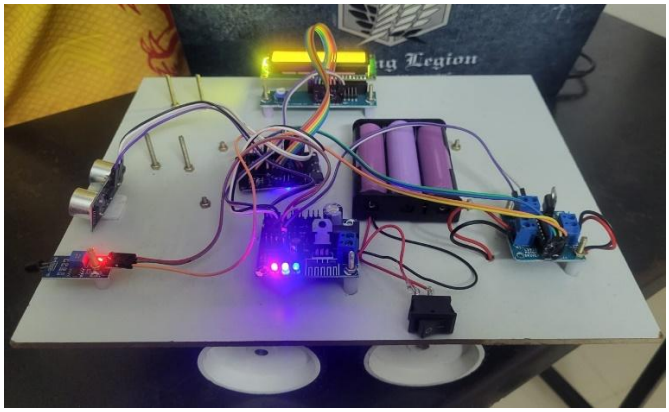


Figure 8.1: Output

8.2 DISCUSSION

The development and testing of the IoT-based surveillance robotic vehicle highlight the practical significance of integrating embedded systems with wireless communication for security and safety applications. The project demonstrates how mobility, sensing, and cloud connectivity can be combined into a unified platform capable of operating in environments that pose substantial risks to human personnel. The overall system performance indicates that such an approach is both technically feasible and operationally beneficial.

One of the most notable observations is the effectiveness of centralized control through the ESP32 microcontroller. By managing sensor inputs, motor operations, and wireless communication within a single processing unit, the system maintains synchronized functionality without unnecessary complexity. This centralized coordination simplifies system architecture while improving reliability. The built-in Wi-Fi capability further enhances adaptability by eliminating the need for additional communication modules.

The integration of multiple sensors contributes significantly to situational awareness. Obstacle detection supports safe navigation, while fire detection enhances early hazard identification. The combination of environmental sensing

with real-time visual feedback creates a layered monitoring strategy, improving the accuracy of threat assessment. Instead of relying solely on numerical data, operators can visually verify conditions, reducing uncertainty and enabling informed decision-making.

Remote accessibility is another key aspect of the system's practical value. Cloud-based monitoring allows users to supervise operations from distant locations, which is particularly important for border areas and disaster zones where physical presence may be limited or dangerous. The ability to control movement and receive instant alerts strengthens responsiveness and reduces dependence on manual patrol routines. This demonstrates the broader capability of IoT technologies in improving in transforming conventional surveillance practices into more dynamic and efficient systems.

The project also reflects the importance of cost-effective design in real-world implementation. By utilizing widely available components such as DC gear motors, L293D motor drivers, and standard sensors, the system remains economically viable while maintaining essential functionality. This balance between affordability and performance supports scalability, making it possible to deploy multiple units across larger regions if required.

Despite the positive outcomes, certain considerations remain for future enhancement. Environmental factors such as extreme weather, signal interference, or rough terrain could influence performance in real-world conditions. Additionally, expanding the system with advanced data analytics or semi-autonomous navigation could further enhance operational capability.

CHAPTER 9

CONCLUSION

The development of the IoT-based surveillance robotic vehicle demonstrates the practical integration of embedded systems, wireless communication, and sensor technology to address challenges in border monitoring and hazardous zone supervision. The project successfully combines mobility, environmental sensing, cloud connectivity, and real-time visualization into a unified platform capable of operating without direct human intervention in high-risk areas. This integration highlights the effectiveness of modern embedded architectures in enhancing security and safety operations.

The system design emphasizes coordinated interaction between hardware components and control logic. The ESP32 microcontroller functions as the central processing unit, managing sensor inputs, wireless communication, and motor control with consistent reliability. The incorporation of fire detection and obstacle sensing improves situational awareness, enabling timely identification of potential threats or environmental hazards. Live video streaming further strengthens monitoring capability by providing visual confirmation of detected events, supporting informed and accurate decision-making.

The robotic mobility achieved through DC gear motors and the L293D motor driver ensures stable navigation across varied surfaces. Remote command execution through IoT connectivity validates the feasibility of operating surveillance systems from distant locations. This feature significantly reduces human exposure to dangerous environments while maintaining operational efficiency. The system’s modular structure also allows straightforward integration of additional components if required, supporting adaptability for different applications.

Another key achievement lies in cost-effectiveness. By utilizing widely available embedded components and standard communication technologies, the project maintains affordability without compromising functionality. This balance between performance and economic feasibility makes the system suitable for scalable deployment in multiple monitoring zones. Reliable power regulation and energy-efficient operation further support extended field usage.

Testing results confirm that the proposed solution provides stable communication, responsive sensor processing, and accurate movement control. The integration of IoT-based alerts and real-time data transmission ensures continuous observation and rapid response capability. Overall, the project validates the concept of using embedded robotic platforms as practical alternatives to conventional surveillance methods.

In conclusion, the system represents a meaningful advancement toward safer and smarter monitoring infrastructure. By minimizing direct human involvement in hazardous conditions and enhancing real-time situational awareness, the project contributes to improved security management. The successful implementation underscores the potential of IoT-enabled robotics in transforming traditional surveillance practices into more dynamic, reliable, and efficient operations.

APPENDIX

```
//military_robo
//Projectiot@123

#include <Arduino.h>
#include <WiFi.h>
#include <ArduinoIoTCloud.h>
#include <Arduino_ConnectionHandler.h>
#include <LiquidCrystal.h>

/* ================= LCD ================= */
LiquidCrystal lcd(2, 15, 19, 18, 5, 4);

/* ================= MOTOR DRIVER ================= */
#define IN1 32
#define IN2 33
#define IN3 21
#define IN4 22

/* ================= ULTRASONIC ================= */
#define TRIG_FRONT 13
#define ECHO_FRONT 12
#define OBSTACLE_THRESHOLD 15 // cm

/* ================= FIRE SENSOR (DIGITAL) ================= */
#define FIRE_PIN 34   // DO pin of fire sensor

/* ================= IOT ================= */
const char DEVICE_LOGIN_NAME[]  = "1465490c-147b-4773-aa30-af4ed3fd2d17";
const char SSID[]               = "projectiot111";
const char PASS[]               = "projectiot111";
```

```
const char DEVICE_KEY[]        = "FqMxJGiMQRGoUXcBaZF5c0gvW";

WiFiConnectionHandler ArduinoIoTPreferredConnection(SSID, PASS);

/* ================= CLOUD VARIABLES ================= */
String contri;     // F, B, L, R, S
int distancei;
bool firei;
bool obstaclei;

/* ================= FUNCTION DECLARATIONS ================= */
void initProperties();
void updateSensors();

/* ================= SETUP ================= */
void setup() {
  Serial.begin(115200);

  // LCD
  lcd.begin(16, 2);
  lcd.print("Railway Robot");
  delay(2000);
  lcd.clear();

  // Motor Pins
  pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);

  // Ultrasonic
  pinMode(TRIG_FRONT, OUTPUT);
  pinMode(ECHO_FRONT, INPUT);

  // Fire Sensor (DIGITAL)
  pinMode(FIRE_PIN, INPUT);
```

```
  // Stop motors initially
  digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);

  // IoT Setup
  initProperties();
  ArduinoCloud.begin(ArduinoIoTPreferredConnection);

  setDebugMessageLevel(2);
  ArduinoCloud.printDebugInfo();
}

/* ================= LOOP ================= */
void loop() {
  ArduinoCloud.update();
  updateSensors();
}

/* ================= SENSOR + LCD ================= */
void updateSensors() {

  long duration, distance;

  // Ultrasonic Read
  digitalWrite(TRIG_FRONT, LOW); delayMicroseconds(2);
  digitalWrite(TRIG_FRONT, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG_FRONT, LOW);

  duration = pulseIn(ECHO_FRONT, HIGH, 30000);
  distance = duration * 0.034 / 2;

  distancei = distance;
  obstaclei = (distance > 0 && distance <= OBSTACLE_THRESHOLD);
```

```
//DIGITAL FIRE SENSOR
int fireState = digitalRead(FIRE_PIN);

// Most fire sensors → LOW = fire detected
firei = (fireState == LOW);

// -------- LCD DISPLAY --------
lcd.clear();

// LINE 1 → CMD + DISTANCE + OBSTACLE
lcd.setCursor(0, 0);

lcd.print("C:");
lcd.print(contri);

lcd.print(" D:");
lcd.print(distancei);
lcd.print("cm ");

if (obstaclei) {
  lcd.print("OD");
} else {
  lcd.print("NO");
}

// LINE 2 → FIRE STATUS
lcd.setCursor(0, 1);

if (firei) {
  lcd.print("FIRE: YES ");
} else {
  lcd.print("FIRE: NO  ");
}
```

```
  // Serial Debug
  Serial.print("CMD: "); Serial.print(contri);
  Serial.print(" | Distance: "); Serial.print(distancei);
  Serial.print(" | Obstacle: "); Serial.print(obstaclei);
  Serial.print(" | Fire: "); Serial.println(firei);

  delay(200);
}

/* ================= IOT CONTROL ================= */
void onContriChange() {

  Serial.print("Command: ");
  Serial.println(contri);

  // STOP if obstacle when moving forward
  if (obstaclei && contri == "F") {
    digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);
    Serial.println("AUTO STOP - Obstacle!");
    return;
  }

  if (contri == "F") {
    // Forward
    digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
    digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
  }
  else if (contri == "B") {
    // Backward
    digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
    digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
  }
```

```
  else if (contri == "L") {
    // Left
    digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
    digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
  }
  else if (contri == "R") {
    // Right
    digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
  }
  else if (contri == "S") {
    // Stop
    digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);
  }
}

/* ================= UNUSED ================= */
void onDistanceiChange() {}
void onFireiChange() {}
void onObstacleiChange() {}

/* ================= IOT PROPERTIES ================= */
void initProperties(){

  ArduinoCloud.setBoardId(DEVICE_LOGIN_NAME);
  ArduinoCloud.setSecretDeviceKey(DEVICE_KEY);

  ArduinoCloud.addProperty(contri, READWRITE, ON_CHANGE, onContriChange);
  ArduinoCloud.addProperty(distancei, READ, ON_CHANGE, NULL);
  ArduinoCloud.addProperty(firei, READ, ON_CHANGE, NULL);
  ArduinoCloud.addProperty(obstaclei, READ, ON_CHANGE, NULL);
}
```

REFERENCE

1) Ahmed, S., Rahman, M., Islam, T. and Hossain, M. (2023) 'IoT-Based Intelligent Surveillance and Monitoring Robot for Security Applications', Proc. Int. Conf. Robotics, Communication and Engineering (ICRCE), pp. 120–125.

2) Hasib, A., Hossain Apu, M.S., Rahman, A., Rana, M.M. and Hossen, R. (2025) 'An IoT-Integrated Autonomous Fire Extinguishing and Surveillance Robot Development Using Smart Camera Module', Proc. 2nd Int. Conf. Next-Generation Computing, IoT and Machine Learning (NCIM), pp. 1–6.

3) Janani, K., Gobhinath, S., Santhosh Kumar, K.V., Roshni, S. and Rajesh, A. (2022) 'Vision Based Surveillance Robot for Military Applications', Proc. 8th Int. Conf. Advanced Computing and Communication Systems (ICACCS), pp. 462–466.

4) Jebastine, J., Nanammal, V. and R J B, V. (2024) 'Enhancing Wireless Surveillance Robot with Real-Time Feedback with AI & IoT Integration', Proc. 9th Int. Conf. Science Technology Engineering and Mathematics (ICONSTEM), pp. 1–10.

5) Karne, R. (2025) 'Embodied AI for Security: Robotic Agents in Surveillance, Disaster Response, and Cyber-Physical Defense', Proc. 10th Int. Conf. Smart Structures and Systems (ICSSS), pp. 1–7.

6) Karmakar, H., Majumder, R., Sarkar, M. and Bhattacharjee, S. (2024) 'Spy–Rover – An IoT Based Surveillance Robot', Proc. Int. Conf. Intelligent Systems and Machine Learning (ISML), pp. 146–151.

7) Kumar, P.S., Naveen, I.G., Parameshachari, B.D. and A C R (2022) 'Military Robot Design and Implementation for Wireless Communication', Proc. IEEE North Karnataka Subsection Flagship Int. Conf. (NKCon), pp. 1–6.

8) Liu, C., Wang, A., Bu, C., Wang, W., Fang, Z. and He, S. (2022) 'Reconstructing Detailed Human Body Using a Viewpoint Auto-Changing

RGB-D Sensor for Rescue Operation', IEEE Sensors Journal, vol. 22, no. 13, pp. 13262–13272.

9) Nair, A.A., Niteesha, C.N.S., Sharma, G. and Nalinakshan, S. (2025) 'Monitoring and Leak Detection of Hazardous Gas Using Internet of Things', Proc. 3rd Int. Conf. Intelligent Data Communication Technologies and Internet of Things (IDCIoT), pp. 739–744.

10) Patel, V., Desai, R., Shah, N. and Trivedi, M. (2024) 'Design of Smart Border Surveillance Robot Using Wireless Communication Technology', Int. J. Advanced Research in Computer and Communication Engineering, vol. 13, no. 2, pp. 45–51.

11) Pawar, K.B., Dharwadkar, N.V., Deshpande, P.A., Honawad, S.K. and Dharmadhikari, P.A. (2021) 'An Android Based Smart Robotic Vehicle for Border Security Surveillance System', Proc. 4th Int. Conf. Computational Intelligence and Communication Technologies (CCICT), pp. 296–301.

12) Rao, B.K., Maddala, A. and Mangali, A. (2025) 'Intelligent Gesture-Based Robotic System with MEMS and RF Technology for Enhanced Motion Control Using ESP32 CAM', Proc. 8th Int. Conf. Computing Methodologies and Communication (ICCMC), pp. 317–323.

13) R, T., S, S. and N, S. (2025) 'IoT-Enabled Battlefield Surveillance Robot with Metal Detection and Real-Time Alerts', Proc. 5th Int. Conf. Ubiquitous Computing and Intelligent Information Systems (ICUIS), pp. 1345–1350.

14) Sharma, P., Gupta, R., Verma, S. and Kaur, H. (2023) 'IoT-Based Smart Surveillance Robot for Border Security Applications', Proc. Int. Conf. Advances in Smart Computing and Artificial Intelligence (ASCAI), pp. 210–215.

15) Singh, I., G V S., Kumar, V., Aarthy, T., Akilan, T. and Sharma, V. (2024) 'Real-Time Monitoring and Surveillance in Hazardous Environments Using IoT-Enabled Robotics', Proc. 1st Int. Conf. Advances in Computing, Communication and Networking (ICAC2N), pp. 1093–1097.

16) Singh, R., Patel, D., Mehta, A. and Joshi, K. (2023) ‘Design and Implementation of Wireless Fire Detection Robot Using IoT’, Int. J. Engineering Research and Technology, vol. 12, no. 4, pp. 556–561.

17) Subramanian, M., Rajkumar, P., Elango, S. and Karthik, R. (2024) ‘Development of IoT Enabled Mobile Robot for Hazardous Environment Monitoring’, Proc. Int. Conf. Emerging Trends in Electronics and Communication (ICETEC), pp. 88–94.

18) Tippannavar, S.S., Puneeth, K.M., Yashwanth, S.D., Madhu Sudan, M.P., Chandrashekar Murthy, B.N. and Vinay Prasad, M.S. (2022) ‘SR2 – Search and Rescue Robot for Saving Endangered Civilians at Hazardous Areas’, Proc. Int. Conf. Disruptive Technologies for Multi-Disciplinary Research and Applications (CENTCON), pp. 21–26.

19) Verma, A., Tiwari, S., Yadav, P. and Chauhan, R. (2022) ‘Autonomous Surveillance Robot with Real-Time Video Transmission Using ESP32’, Proc. IEEE Int. Conf. Communication, Control and Information Sciences (ICCIS), pp. 345–350.

20) Zhang, Y., Li, X., Chen, H. and Wang, J. (2023) ‘Edge-Enabled IoT Robotic System for Intelligent Security Monitoring’, IEEE Access, vol. 11, pp. 78214–78225.